DATABASE MANAGEMENT SYSTEMS

CT 652 | BE / BCT | Year III / Part II

Tribhuvan University — Institute of Engineering

CHAPTER 5: QUERY PROCESSING & OPTIMIZATION

Question Bank with Comprehensive Answers — Processing Steps, Algorithms, Heuristic & Cost-Based Optimization, Join Algorithms

SYLLABUS OVERVIEW — Unit 5

Unit	Topic	Hours	Marks
1	Introduction	3 hrs	4
2	Data Models	7 hrs	12
3	Relational Languages and Relational Model	7 hrs	12
4	Database Constraints and Normalization	6 hrs	12
5	Query Processing and Optimization	4 hrs	8
6	File Structure and Hashing	4 hrs	8
7	Transactions Processing and Concurrency Control	6 hrs	12
8	Crash Recovery	4 hrs	6
9	Advanced Database Concepts	4 hrs	6
Total		**45 hrs**	**80**

Chapter 5 Topics (4 Hours | 8 Marks)

- 5.1 Query Processing Steps
- 5.2 Translating SQL to Relational Algebra
- 5.3 Algorithms for Relational Algebra Operations (selection, join)
- 5.4 Query Optimization — heuristic and cost-based
- 5.5 Cost Estimation (disk I/O model, statistics)
- 5.6 Join Ordering
- 5.7 Pipelining in Query Evaluation

CHAPTER 5 — Questions & Comprehensive Answers

Q: Explain the steps involved in query processing with a neat diagram.
Asked: ALL years 2070–2082 — asked every single exam | Marks: [4]

Answer

Query Processing

Query processing refers to the range of activities involved in extracting data from a database in response to a user query. It transforms a high-level query (SQL) into an efficient execution strategy and executes it to produce the result. The DBMS goes through several well-defined steps between receiving an SQL query and returning the answer.

Query Processing Pipeline — Diagram

```
┌──────────────────────────────────────────────────────────┐
│                    USER / APPLICATION                    │
│            SQL Query: SELECT name FROM ...               │
└──────────────────────────────────────────────────────────┘
                              │
                              │  SQL string
                              ▼
┌──────────────────────────────────────────────────────────┐
│               1. SCANNER & PARSER                        │
│   Lexical analysis (tokens) + Syntax checking            │
│   Output: Parse tree / Abstract Syntax Tree (AST)        │
└──────────────────────────────────────────────────────────┘
                              │
                              │  Parse tree
                              ▼
┌──────────────────────────────────────────────────────────┐
│               2. SEMANTIC ANALYZER                       │
│   Validates table/column names against catalog           │
│   Output: Annotated query / Relational Algebra expr      │
└──────────────────────────────────────────────────────────┘
                              │
                              │  Relational Algebra expression
                              ▼
┌──────────────────────────────────────────────────────────┐
│          3. QUERY OPTIMIZER                              │
│   Generates candidate execution plans                    │
│   Estimates cost of each plan                            │
│   Uses heuristic rules + statistics                      │
│   Output: Optimal (or near-optimal) query plan           │
└──────────────────────────────────────────────────────────┘
                              │
                              │  Query execution plan
                              ▼
┌──────────────────────────────────────────────────────────┐
│          4. CODE GENERATOR / PLAN EXECUTOR               │
│   Translates plan into low-level execution code          │
│   Accesses data via storage/buffer manager               │
│   Output: Result tuples                                  │
└──────────────────────────────────────────────────────────┘
                              │
                              │  Query result
                              ▼
┌──────────────────────────────────────────────────────────┐
```

```
│                    USER / APPLICATION                    │
│                Result set returned                       │
└──────────────────────────────────────────────────────────┘
```

Detailed Description of Each Step

Step 1: Scanner and Parser (Lexical & Syntax Analysis)

The scanner performs lexical analysis — it tokenizes the SQL query string into keywords (SELECT, FROM, WHERE), identifiers (table/column names), operators, and literals. The parser performs syntax analysis — it checks whether the token sequence conforms to SQL grammar rules and builds a parse tree (also called an Abstract Syntax Tree, AST).

- Detects syntax errors (e.g., missing semicolons, mismatched parentheses)
- Output: A parse tree representing the syntactic structure of the query
- Example: SELECT name FROM Employee WHERE salary > 50000 → parse tree with SELECT node, FROM clause node, WHERE clause node

Step 2: Semantic Analyzer (Query Validation)

The semantic analyzer checks the parse tree against the database catalog (data dictionary). It validates that all referenced tables, columns, and data types exist and are used correctly. It also resolves attribute references and checks access permissions.

- Validates table names exist in the schema
- Validates column names belong to the referenced tables
- Checks data type compatibility in expressions
- Output: Annotated relational algebra expression (initial query plan)

Step 3: Query Optimizer

The query optimizer is the most critical and complex component of query processing. It takes the relational algebra expression from the semantic analyzer and transforms it into an efficient execution plan.

- Generates multiple equivalent execution plans using algebraic equivalence rules
- Estimates the cost (primarily disk I/Os) of each candidate plan
- Uses catalog statistics (relation sizes, distinct values, histograms) for cost estimation
- Selects the plan with minimum estimated cost
- Two main approaches: heuristic (rule-based) and cost-based
- Output: Annotated query execution plan (query tree with chosen algorithms)

Step 4: Code Generator / Query Executor

The code generator translates the optimized query plan into actual executable code (or directly executes it via an interpreter). It interacts with the storage manager, buffer pool manager, and index structures to physically retrieve data.

- Calls appropriate algorithms for each relational algebra operation
- Manages buffer pages, reads/writes to disk
- Handles pipelining between operations where possible
- Output: The actual result tuples returned to the user/application

Step	Component	Input	Output
1	Scanner & Parser	SQL string	Parse tree (AST)
2	Semantic Analyzer	Parse tree + catalog	Relational Algebra expression
3	Query Optimizer	RA expression + statistics	Optimal execution plan
4	Code Generator / Executor	Execution plan	Result tuples

Q: Translate the SQL query to a relational algebra expression and show how it is optimized using a query tree. SQL: SELECT name FROM Employee WHERE dept='Sales' AND salary>50000

Asked: 2080 Ashwin; 2079 Ashwin; 2073 Magh (all similar) | Marks: [4+4]

Answer

SQL to Relational Algebra Translation

Given SQL:

```
SELECT name
FROM   Employee
WHERE  dept = 'Sales' AND salary > 50000
```

Step 1: Direct (Naive) Translation to Relational Algebra

The straightforward translation applies all clauses mechanically:

```
π name ( σ dept='Sales' ∧ salary>50000 ( Employee ) )

Read: Project 'name' from the selection of Employee tuples where
      dept='Sales' AND salary>50000
```

Step 2: Initial Query Tree (Unoptimized)

```
                π name
                  |
σ dept='Sales' ∧ salary>50000
                  |
              Employee
```

Step 3: Apply Heuristic Optimization Rules

For a single-table query, the initial translation is already quite good. For multi-table queries, optimization is more significant. Let's extend with a join example:

```
SELECT e.name, d.dname
FROM   Employee e, Department d
WHERE  e.dept = d.deptno
```

```
AND    d.dname = 'Sales'
AND    e.salary > 50000
```

```
Naive RA: π e.name,d.dname ( σ e.dept=d.deptno ∧ d.dname='Sales' ∧
e.salary>50000
                              ( Employee × Department ) )
```

Step 4: Optimized Query Tree — push selections down

```
                                π e.name, d.dname
                                          |
                              (e.dept = d.deptno)
                               /                  \
                  σ salary>50000           σ dname='Sales'
                          |                        |
                     Employee                 Department
```

Optimization applied: Push σ salary>50000 below the join to filter Employee early (reduces join input size). Push σ dname='Sales' below the join to filter Department early. The join now operates on much smaller relations, reducing disk I/Os significantly.

Step 5: Further Optimization — push projections down

```
                                π e.name, d.dname
                                          |
                              (e.dept = d.deptno)
                              /                    \
                  π e.name, e.dept          π d.deptno, d.dname
                          |                            |
                  σ salary>50000               σ dname='Sales'
                          |                            |
                     Employee                     Department
```

Projections eliminate unnecessary columns early, further reducing the size of intermediate results passed to the join.

Q: What is query optimization? Why is it needed? Explain cost-based and heuristic optimization. Distinguish between the two approaches.

Asked: 2081 Ashwin; 2079 Jestha; 2074; 2073 Magh; 2072 Magh; 2070–2082 | Marks: [4–6]

Answer

Query Optimization

Query optimization is the process by which a DBMS selects the most efficient strategy (execution plan) for executing a given query. Since a single SQL query can be executed in many equivalent ways (different join orders, different algorithms for each operation, different access paths), the optimizer's job is to find the plan with the lowest estimated execution cost.

Why Query Optimization is Needed

- Multiple equivalent plans: A simple 3-table join can have 5+ different join orderings, each with 3+ algorithm choices per join — resulting in hundreds of possible plans
- Performance difference is dramatic: A bad plan might take minutes or hours; an optimal plan might take milliseconds — for the same query on the same data
- User writes what, not how: SQL is declarative — the user says WHAT data they want, not HOW to retrieve it; the DBMS must decide the how
- Data characteristics change: Table sizes, data distributions, and available indexes change over time; the optimizer adapts based on up-to-date statistics
- Hardware awareness: The optimizer considers buffer pool size, disk access patterns, and I/O costs

1. Heuristic (Rule-Based / Algebraic) Optimization

Heuristic optimization applies a fixed set of transformation rules to the query's relational algebra expression, without estimating actual costs. These rules are based on the observation that certain transformations almost always improve performance.

- Also called: algebraic optimization or rule-based optimization
- Uses algebraic equivalence rules to rewrite the query
- Does NOT require statistical information about data
- Faster to apply, but may not always find the globally optimal plan
- Used as a preprocessing step before cost-based optimization

2. Cost-Based Optimization

Cost-based optimization enumerates multiple candidate execution plans (or a representative subset), estimates the cost of each using statistical information about the data, and selects the plan with minimum estimated cost.

- Uses catalog statistics: number of tuples, number of pages, number of distinct values, histograms
- Cost metric: primarily number of disk I/O operations; sometimes also CPU cost and network cost
- More accurate than heuristic but computationally expensive for complex queries
- Uses dynamic programming or greedy algorithms to manage plan enumeration
- Considers access paths: sequential scan, B+ tree index, hash index

Aspect	Heuristic Optimization	Cost-Based Optimization
Approach	Applies transformation rules (algebraic equivalences)	Enumerates plans, estimates and compares costs
Statistics needed	No — rule-based; no data statistics required	Yes — requires relation sizes, distributions, etc.
Accuracy	May not find globally optimal plan	Finds near-optimal plan based on estimated costs
Speed	Fast (rule application is quick)	Slower (must estimate cost of many candidate plans)
When to use	Pre-processing step; simple queries	Complex queries with multiple joins

Examples of rules/techniques	Push selections down, push projections down	Dynamic programming for join ordering, selectivity estimation
Industry practice	Used first to simplify the expression	Then applied to select best among remaining alternatives

Q: Describe different cost measures for query evaluation. Explain disk I/O cost model and relation statistics used by the optimizer.

Asked: 2081 Chaitra; 2078 Baishakh; 2075 Bhadra; 2071 Magh; 2070 Magh | Marks: [4]

Answer

Cost Measures for Query Evaluation

The cost of executing a query plan is measured in terms of the resources consumed. The dominant cost in traditional disk-based database systems is the number of disk I/O operations (block transfers and seeks), since disk access is orders of magnitude slower than memory access.

Cost Components

1. Disk I/O Cost (Primary Cost Metric)

- Block transfer cost: time to read/write one disk block (page) — typically denoted t_T
- Seek time: time for disk head to move to the correct track — denoted t_S
- Total I/O cost for b block transfers and S seeks = b × t_T + S × t_S
- Typical values: t_T = 0.1 ms (SSD) to 4 ms (HDD); t_S ≈ 4–10 ms (HDD)

2. CPU Cost

Cost of in-memory operations (comparisons, hash computations, sorting). Usually secondary to disk I/O in traditional systems, but significant in in-memory databases.

3. Network Cost

In distributed database systems, the cost of transmitting data between nodes. Measured in bytes or message count.

4. Memory (Buffer) Usage

The number of buffer pages (frames) available affects algorithm choices. More memory allows more data to be processed in-memory, reducing disk I/Os.

Key Notation

```
n_r   = number of tuples (cardinality) of relation r
b_r   = number of disk blocks containing tuples of r
l_r   = size of each tuple in bytes
f_r   = blocking factor = tuples per block = □block_size / l_r□
V(A,r) = number of distinct values of attribute A in relation r
```

```
          (V(A,r) = selectivity-related statistic)
M     = number of buffer pages available in memory
h_i   = height of B+ tree index i
```

Relation Statistics in the Catalog

The DBMS system catalog maintains statistics that the optimizer uses for cost estimation. These are updated periodically (e.g., via ANALYZE command in PostgreSQL).

Statistic	Description	Used For
n_r (cardinality)	Total number of tuples in relation r	Estimating output size of selections, joins
b_r (block count)	Total number of disk blocks used by r	Sequential scan cost = b_r block reads
f_r (blocking factor)	Tuples per block; f_r = □block_size/l_r□	Deriving b_r from n_r
V(A,r) (distinct values)	Number of distinct values of attribute A	Selectivity of equijoin/selection; n_r/V(A,r) = avg tuples per value
min(A,r), max(A,r)	Min and max value of attribute A	Range query selectivity estimation
Histogram	Distribution of values of A in r	More accurate selectivity for skewed distributions
Index statistics	Height, number of leaf pages of index i	Index scan cost estimation

Selectivity Estimation

The selectivity of a predicate is the fraction of tuples that satisfy it. It determines the size of the output of a selection operation.

```
For equality: σ A=v (r)
  Estimated output tuples = n_r / V(A,r)
  Example: 10000 tuples, V(dept,r)=50 depts → σ dept='Sales' → ~200 tuples

For range: σ A≥v (r)
  Estimated output = n_r × (max(A,r) - v) / (max(A,r) - min(A,r))

For join: r □ s  on r.A = s.B
  Estimated output = (n_r × n_s) / max(V(A,r), V(B,s))
  Special case: if A is PK of r, join output ≤ n_s
```

Q: What is heuristic optimization? Explain the key heuristic rules (equivalence rules) used in algebraic optimization with examples.

Asked: 2082 Kartik; 2079 Chaitra; 2078 Chaitra; 2075 Baisakh; 2073; 2072 Ashwin; 2071; 2070 (all years) | Marks: [4–6]

Answer

Heuristic (Algebraic) Optimization

Heuristic optimization rewrites a relational algebra expression using algebraic equivalence rules — transformations that produce an equivalent expression (same result) but (usually) more efficiently. The key insight is that pushing selections and projections as early (as close to the leaves) as possible in the query tree dramatically reduces the size of intermediate results.

Algebraic Equivalence Rules

Two relational algebra expressions are equivalent if they produce the same set of tuples on every valid database instance. The following rules are used for optimization:

Rule 1: Cascade of Selections (Conjunctive Selection Splitting)

```
σ p1 ∧ p2 (E)  ≡  σ p1 (σ p2 (E))

A selection with a conjunction of conditions can be split into
a series of individual selections.

Example: σ dept='Sales' ∧ salary>50000 (Employee)
       ≡ σ dept='Sales' (σ salary>50000 (Employee))

Usage: Allows each individual selection to be pushed down independently.
```

Rule 2: Commutativity of Selection

```
σ p1 (σ p2 (E))  ≡  σ p2 (σ p1 (E))

Order of selections does not matter.
Apply the MORE SELECTIVE predicate first to reduce tuples faster.
```

Rule 3: Cascade of Projections

```
π L1 (π L2 (... (π Ln (E)) ...))  ≡  π L1 (E)
  (where L1 □ L2 □ ... □ Ln)

Only the outermost projection matters — inner projections are redundant.
BUT: intermediate projections can eliminate unneeded columns early.
```

Rule 4: Commuting Selection with Projection

```
π A1,...,An (σ p (E))  ≡  σ p (π A1,...,An (E))
  ONLY if predicate p involves only attributes A1,...,An

Allows projection to be pushed below selection when safe.
```

Rule 5: Commutativity of Natural Join / Cartesian Product

```
E1 ⋈ E2  ≡  E2 ⋈ E1
E1 × E2  ≡  E2 × E1
```

```
Join order can be swapped. Used for join ordering optimization.
```

Rule 6: Associativity of Natural Join

```
(E1 ⋈ E2) ⋈ E3  ≡  E1 ⋈ (E2 ⋈ E3)

Allows reordering of joins. Critical for join ordering optimization.
With commutativity: any join order is achievable.
```

Rule 7: Commuting Selection with Join (Push Selection Down)

```
If predicate p involves ONLY attributes of E1:
  σ p (E1 ⋈ E2)  ≡  (σ p (E1)) □ E2

If predicate p involves ONLY attributes of E2:
  σ p (E1 ⋈ E2)  ≡  E1 ⋈ (σ p (E2))

If predicate p = p1 ∧ p2 where p1 involves only E1 and p2 only E2:
  σ p1 ∧ p2 (E1 ⋈ E2)  ≡  (σ p1 (E1)) □ (σ p2 (E2))

*** THIS IS THE MOST IMPORTANT OPTIMIZATION RULE ***
Push selections as close to the leaves as possible to reduce
the size of relations before performing expensive join operations.
```

Rule 8: Commuting Projection with Join

```
If L = L1 ∪ L2 (L1 ⊆ attrs(E1), L2 ⊆ attrs(E2)):
  π L (E1 ⋈ E2)  ≡  (π L1∪(join attrs of E1) (E1)) □ (π L2∪(join attrs of E2)
(E2))

Push projections down through joins, keeping only needed attributes.
This reduces the width (number of columns) of intermediate results.
```

Rule 9: Commuting Selection with Set Operations

```
σ p (E1 ∪ E2)  ≡  (σ p (E1)) ∪ (σ p (E2))
σ p (E1 ∩ E2)  ≡  (σ p (E1)) ∩ (σ p (E2))
σ p (E1 − E2)  ≡  (σ p (E1)) − (σ p (E2))

Selection distributes over union, intersection, and set difference.
```

Rule 10: Selection/Join Combination (Converting θ-join)

```
σ p (E1 × E2)  ≡  E1 □p E2

A selection over a Cartesian product is equivalent to a theta-join.
Always prefer theta-join over Cartesian product + selection.
```

<u>General Heuristic Strategy</u>

- Step 1: Break conjunctive selections into a cascade of individual selections (Rule 1)
- Step 2: Push each selection as far down the query tree as possible (Rule 7 — most critical)
- Step 3: Replace Cartesian products + selections with join operations (Rule 10)
- Step 4: Determine the most efficient join ordering (smallest intermediate results first)
- Step 5: Push projections down to eliminate unused columns early (Rules 3, 8)
- Step 6: Combine adjacent selections and projections where possible

Worked Example — Heuristic Optimization:

```
Query: Find names of employees in the 'Sales' department
       who earn more than any employee in 'Research' department

SQL:
SELECT e1.name
FROM   Employee e1, Department d1
WHERE  e1.dept_id = d1.dept_id
AND    d1.dname = 'Sales'
AND    e1.salary > (SELECT MAX(e2.salary)
                    FROM Employee e2, Department d2
                    WHERE e2.dept_id = d2.dept_id
                    AND   d2.dname = 'Research')
```

```
Naive RA (simplified):
π name ( σ dname='Sales' ∧ e.salary > MAX_research_sal
          ( Employee × Department ) )

After heuristic optimization:
1. Split σ: σ dname='Sales' AND σ e.salary > computed_value
2. Push σ dname='Sales' below join → apply to Department first
3. Push σ e.dept_id=d.dept_id to form join
4. Push π name to eliminate unneeded columns early

Optimized:
π name ( (σ salary > subquery_result (Employee))
          □(dept_id)  (σ dname='Sales' (Department)) )
```

Q: What is cost-based query optimization? Explain the plan enumeration approach and dynamic programming for join ordering.

Asked: 2081 Ashwin; 2079 Jestha; 2073 Magh; 2072 Magh; 2070 Magh | Marks: [4–6]

Answer

Cost-Based Query Optimization

In cost-based optimization, the query optimizer generates multiple candidate execution plans, estimates the cost of each plan using statistical information about the database, and chooses the plan with the minimum estimated cost.

Plan Enumeration

For a query with n relations being joined, there are potentially n! different join orderings, and each join can be implemented with multiple algorithms. For n=10 relations, this is 3,628,800 orderings — exhaustive enumeration is impractical.

Left-Deep Join Trees

In practice, optimizers restrict enumeration to left-deep join trees, where the right operand of every join is a base relation (not a sub-join). This restriction reduces the search space to n! orderings (still large, but manageable with dynamic programming).

```
                        Left-deep tree for r1 □ r2 □ r3 □ r4:

                                          □
                                        / \
                                      □   r4
                                     / \
                                    □   r3
                                   / \
                                 r1  r2

               Right operand is always a base table — allows pipelining
```

Dynamic Programming for Join Ordering

The optimal join ordering is found using dynamic programming, which avoids redundant computation by reusing already-computed subresults.

```
Algorithm: Optimal join ordering by dynamic programming

Input:  Relations r1, r2, ..., rn; cost function cost(plan)
Output: Optimal join ordering and its cost

Initialization:
  For each relation ri:
    BestPlan({ri}) = (sequential scan or best index scan of ri)
    Cost({ri}) = cost of scanning ri alone

Main loop: For subset size s = 2 to n:
  For each subset S of size s:
    BestPlan(S) = ∞
    For each relation ri □ S:
      S' = S - {ri}
      plan = BestPlan(S') □ ri   (use best algorithm for this join)
      cost = Cost(BestPlan(S')) + cost(BestPlan(S') □ ri)
      If cost < BestPlan(S).cost:
          BestPlan(S) = plan
          Cost(S) = cost

Answer: BestPlan({r1, r2, ..., rn}) with Cost({r1,...,rn})

Time complexity: O(3^n) subsets; practical for n ≤ 10-15 relations
```

Interesting Orders

Some physical properties of the output (e.g., being sorted on a particular attribute) can reduce the cost of subsequent operations. An optimizer may retain sub-optimal plans with interesting orders if they

avoid expensive sort operations later — this is the concept of interesting orderings (Selinger et al., 1979).

Selectivity and Join Size Estimation

```
Selection selectivity of σ A=v(r):  sel = 1/V(A,r)
Estimated output tuples of σ A=v(r) = n_r / V(A,r)

Join size estimate for r □ s on r.A = s.B:
  Estimated tuples = (n_r × n_s) / max(V(A,r), V(B,s))
  Special: if A is PK of r → estimate = n_s
           if B is PK of s → estimate = n_r

Example:
  Employee: n=10000, V(dept_id)=50
  Department: n=50
  Join Employee □ Department on dept_id:
  Estimate = (10000 × 50) / max(50,50) = 500000/50 = 10000 tuples
```

Q: Explain algorithms for selection operation: linear scan, binary search, primary index scan, and secondary index scan. Give the cost of each.

Asked: 2078 Poush; 2074; 2072 Magh | Marks: [4]

Answer

Algorithms for Selection Operation

A selection operation σ p (r) retrieves all tuples of r satisfying predicate p. The choice of algorithm depends on the type of predicate, available indexes, and whether the file is sorted.

Algorithm 1: Linear Scan (Full Table Scan)

Scan every block of the relation and check each tuple against the predicate. Works for any predicate, regardless of indexes or sort order.

```
Cost: b_r block transfers + 1 seek
  (b_r = number of blocks in relation r)

If only one tuple satisfies (equality on key):
  Average cost = b_r/2 block transfers (stop early on average)

Example: r has 1000 blocks → cost ≈ 1000 block reads
Best for: small relations, no useful index, very low selectivity
```

Algorithm 2: Binary Search

Applicable only when the relation is stored sorted on the search attribute. Eliminates half the remaining search space at each step.

```
Cost: □log2(b_r)□ block transfers + □log2(b_r)□ seeks
```

```
      for the first matching block
      then additional blocks for range/duplicate values

For equality on unique key: ⌈log2(b_r)⌉ block transfers
For equality on non-unique: ⌈log2(b_r)⌉ + extra blocks with same value

Example: r has 1000 blocks → ⌈log2(1000)⌉ = 10 block reads
Best for: sorted file, equality or range query on sort key
```

Algorithm 3: Primary Index (B+ Tree on Search Key)

Use a primary B+ tree index on the search attribute. The index provides a pointer to the first block containing the matching tuples; then follow the pointer into the data file.

```
For equality on primary key (unique):
  Cost = h_i + 1 block transfers + (h_i + 1) seeks
  where h_i = height of B+ tree (typically 3-4 for large tables)

For equality on non-unique primary index:
  Cost = h_i + b block transfers (b = extra blocks with same value)

For range query σ A≥v (r):
  Find first matching entry in index; scan forward in leaf pages
  Cost = h_i + number of matching blocks

Example: h_i=3, 1 matching block → cost = 3+1 = 4 block transfers
  vs linear scan of 1000 blocks → 96% improvement!
```

Algorithm 4: Secondary Index (B+ Tree on Non-Search-Key)

Use a secondary index (on a non-sort-order attribute). The index provides pointers to individual records, but these records may be scattered across many blocks.

```
For equality on candidate key (unique):
  Cost = h_i + 1 block transfers

For equality on non-key (non-unique):
  Cost = h_i + number_of_matching_tuples block transfers
  (worst case: each matching tuple in a different block!)

Example: σ dept='Sales' on 10000-tuple table, 200 Sales employees
  Secondary index: h_i=3, up to 200 block reads (one per tuple)
  vs primary index or linear scan: might be WORSE than linear scan
  for high-selectivity queries (this is the 'index paradox')

Best for: Low-selectivity queries (few matching tuples) with secondary index.
Avoid for: High selectivity (many matching tuples) — linear scan may be
faster.
```

Algorithm	Applicable When	Cost (block transfers)	Best For
Linear Scan	Always (any predicate)	b_r	No index; small relations; very low selectivity

Binary Search	File is sorted on search attr	□log2(b_r)□	Sorted file, equality or range on sort key
Primary B+ Index	B+ index exists on search attr	h_i + 1 (unique key)	Unique key lookup; range queries with index
Secondary B+ Index	Secondary index on any attr	h_i + matching tuples	Equality on non-key, few matching tuples

---

Q: Explain the Nested Loop Join algorithms (basic NLJ, block NLJ, indexed NLJ) with their cost formulas.
Asked: 2081 Ashwin; 2080; 2077; 2076; 2075 Baisakh; 2073 Magh; 2072 Magh; 2071; 2070 (all years) | Marks: [4–6]

Answer

Nested Loop Join (NLJ) Algorithms

Nested loop join algorithms compute r □ s by comparing tuples from the outer relation r against tuples of the inner relation s, one by one. The relation r is the outer relation and s is the inner relation.

Algorithm 1: Basic (Tuple-at-a-Time) Nested Loop Join

```
for each tuple t_r in r do              -- outer loop: n_r iterations
    for each tuple t_s in s do          -- inner loop: n_s iterations
        if t_r and t_s satisfy join condition:
            output (t_r, t_s)
```

```
Cost: n_r × b_s + b_r   block transfers
      n_r + b_r          seeks

where: n_r = number of tuples in outer relation r
       b_r = number of blocks in r
       b_s = number of blocks in s

Example: r has 5000 tuples, 100 blocks; s has 10000 tuples, 400 blocks
  Using r as outer: 5000 × 400 + 100 = 2,000,100 block transfers
  Using s as outer: 10000 × 100 + 400 = 1,000,400 block transfers
  → Choose smaller relation as outer to minimize cost

Worst case (1 buffer page): O(n_r × b_s)
Best case (s fits in memory): b_r + b_s
```

Algorithm 2: Block Nested Loop Join (BNLJ)

Instead of reading the inner relation once per outer tuple, read the inner relation once per outer block. Uses available buffer memory more efficiently.

```
for each BLOCK B_r of r do              -- outer loop: b_r iterations
    for each BLOCK B_s of s do          -- inner loop: b_s iterations
        for each tuple t_r in B_r do
            for each tuple t_s in B_s do
```

```
                if t_r and t_s satisfy join condition:
                    output (t_r, t_s)
```

```
Cost: b_r × b_s + b_r   block transfers
      2 × b_r            seeks (1 seek per outer block + 1 for inner scan)

Example: r: 100 blocks; s: 400 blocks
  BNLJ: 100 × 400 + 100 = 40,100 block transfers
  vs Basic NLJ: 5000 × 400 + 100 = 2,000,100 block transfers
  → 50x improvement by using block-level outer loop

With M buffer pages (M-2 pages for outer blocks):
  Cost = □b_r / (M-2)□ × b_s + b_r   block transfers
  → Use as many buffer pages as possible for outer relation
  → If M-2 ≥ b_r: inner relation scanned only once → b_r + b_s transfers
```

Algorithm 3: Indexed Nested Loop Join

If an index exists on the join attribute of the inner relation s, use the index to look up matching tuples of s for each tuple of r, instead of scanning all of s.

```
for each tuple t_r in r do              -- outer loop: n_r iterations
    use index on s.join_attr to find    -- index lookup for each t_r
    all tuples t_s where t_s.B = t_r.A
    for each such tuple t_s:
        output (t_r, t_s)
```

```
Cost: b_r + n_r × c   block transfers
      b_r + n_r × c   seeks

where c = cost of a single index probe on s:
  Primary B+ tree: c = h_i + 1
  Secondary B+ tree: c = h_i + matching tuples in s

Example: r: 5000 tuples, 100 blocks; s has B+ tree on join attr, h_i=3
  If join is on PK of s (each lookup returns 1 tuple): c = h_i + 1 = 4
  Cost = 100 + 5000 × 4 = 20,100 block transfers
  vs Block NLJ: 40,100 — about 2× improvement with index

Best when: r is small (outer) and s has an index on the join attribute.
```

Algorithm	Cost (block transfers)	When to Use
Basic NLJ	n_r × b_s + b_r	Small inner relation or no memory; simple but expensive
Block NLJ (BNLJ)	□b_r/(M-2)□ × b_s + b_r	No index; use as many buffer pages as possible for outer
Indexed NLJ	b_r + n_r × c	Index exists on inner relation's join attribute; outer is small

Q: Explain the Sort-Merge Join algorithm in detail with its phases, pseudocode, and cost formula. Compare with Nested Loop Join.

Asked: 2081 Ashwin; 2080; 2079 Jestha; 2077; 2076; 2075 Bhadra; 2074; 2072 Ashwin; 2071; 2070 (all years) | Marks: [4]

Answer

Sort-Merge Join (SMJ)

Sort-Merge Join is a join algorithm that first sorts both relations on the join attribute, then merges the sorted relations by scanning them simultaneously. It is particularly efficient when both relations are already sorted or when large relations must be joined and no index is available.

Algorithm: Two Phases

Phase 1: Sort

```
Sort relation r on join attribute A
Sort relation s on join attribute B

External merge sort is used when relations don't fit in memory.
Cost of sorting r: 2 × b_r × □log_{M-1}(b_r/M)□ + b_r   (approx)
  (where M = buffer pages available)
  Simplified: ≈ 2 × b_r + 2 × b_s block transfers for sorting both
```

Phase 2: Merge

```
p_r = first tuple of sorted r
p_s = first tuple of sorted s

while p_r != EOF and p_s != EOF:
    if p_r.A < p_s.B:  advance p_r
    elif p_r.A > p_s.B:  advance p_s
    else:  // p_r.A == p_s.B
        // Match found: output all combinations
        mark position in s (for duplicates in r)
        while p_r.A == marked.B:  // advance through all matching r tuples
            reset p_s to marked position
            while p_s.B == p_r.A:  // advance through all matching s tuples
                output (p_r, p_s)
                advance p_s
            advance p_r
```

Cost Analysis

```
Sort phase cost:
  Sort r:  2 × b_r × □log_{M-1}□b_r/M□□ + b_r   ≈  2 × b_r × passes
  Sort s:  similar
  For 2 passes (M large enough): sort cost ≈ 4(b_r + b_s)

Merge phase cost: b_r + b_s   (scan both sorted files once)

Total cost (if sorting needed):
```

```
  ≈ 3(b_r + b_s)   block transfers   (sort in 1 pass + merge)

If both relations already sorted: b_r + b_s   (only merge phase)

Example: r: 100 blocks, s: 400 blocks, M=10 buffer pages
  Sort cost (1 pass): 2×100 + 2×400 = 1000
  Merge cost: 100 + 400 = 500
  Total: ~1500 block transfers
  vs BNLJ: 100×400+100 = 40,100 — SMJ is ~27× faster!
```

Sort-Merge Join vs Nested Loop Join

Aspect	Sort-Merge Join (SMJ)	Block Nested Loop Join (BNLJ)
Cost (unsorted)	3(b_r+b_s) approx (sort+merge)	□b_r/(M-2)□ × b_s + b_r
Cost (pre-sorted)	b_r + b_s (merge only)	No benefit from sorted order
Memory requirement	Must buffer sort runs + merge buffers	Works with as few as 3 buffer pages
Output order	Produces sorted output (free!)	No ordering guarantee
Best when	Large relations, subsequent sort needed, or pre-sorted	Memory limited, or one relation is small
Index usage	Does not require index	Indexed NLJ benefits from index
Scalability	O((b_r+b_s)log(b_r+b_s)) sort + O(b_r+b_s) merge	O(b_r × b_s) worst case

---

Q: Explain the Hash Join algorithm with its phases (build and probe). Give the cost analysis.

Asked: 2081 Ashwin; 2078 Baishakh; 2075 Bhadra; 2073 Magh; 2071 Magh | Marks: [4]

Answer

Hash Join Algorithm

Hash join uses a hash function on the join attribute to partition both relations into buckets. Tuples from both relations that could possibly join (having the same join attribute value) will be in corresponding buckets. Only corresponding bucket pairs need to be compared, dramatically reducing the comparison work.

Algorithm: Two Phases

Phase 1: Build Phase (Hashing the Build Relation)

```
Choose the SMALLER relation as the build relation (say r)
Apply hash function h1 on join attribute A of r

Partition r into h buckets: r0, r1, r2, ..., rh-1
```

```
  Bucket ri = { t □ r | h1(t.A) = i }

For each partition ri:
    Load ri into memory (hash table using a second hash function h2)

Goal: Each partition must fit in memory for the probe phase.
If M buffer pages available:
  Use h ≈ M-1 partitions (1 output buffer per partition)
  Each partition of r should have size ≈ b_r/h ≤ M pages
```

Phase 2: Probe Phase (Probing the Probe Relation)

```
Apply the SAME hash function h1 on join attribute B of s (probe relation)

For i = 0 to h-1:
    Load partition si (from s) into memory
    For each tuple t_s in si:
        Probe the in-memory hash table of ri using h2(t_s.B)
        For each matching t_r (where t_r.A == t_s.B):
            output (t_r, t_s)
```

Cost Analysis

```
Partition phase:
  Read r:   b_r block transfers + b_r/M seeks
  Write r partitions:  b_r block transfers
  Read s:   b_s block transfers
  Write s partitions:  b_s block transfers
  Total partition: 2(b_r + b_s) block transfers

Build & Probe phase:
  Read all partitions of r: b_r block transfers
  Read all partitions of s: b_s block transfers
  Total build+probe: b_r + b_s block transfers

Total Hash Join Cost: 3(b_r + b_s) block transfers
  (same formula as Sort-Merge Join, but no sorting overhead)

Condition: Build relation partition must fit in memory:
  b_r / h ≤ M  →  h ≥ b_r/M  →  M ≥ √b_r  (approximately)

Example: r: 100 blocks, s: 400 blocks, M=12
  Total = 3 × (100+400) = 1500 block transfers
  Same as SMJ for this example
```

Hash Join vs Sort-Merge Join vs BNLJ

Aspect	Hash Join	Sort-Merge Join	Block NLJ
Cost	3(b_r + b_s)	3(b_r + b_s)	□b_r/(M-2)□×b_s+b_r
Memory req.	M ≥ √min(b_r,b_s)	M ≥ 3 (but more = faster)	M ≥ 3
Output order	Unsorted	Sorted on join attr	Unsorted

Pre-sorted input	No benefit	Saves sort phase cost	No benefit
Skew problem	Uneven hash distribution hurts	Duplicate join values handled	N/A
Best for	Large equi-joins, no sort needed	When sorted output needed, or pre-sorted	Small inner; memory limited

---

Q: What is join ordering? How does it affect query performance? Explain the dynamic programming approach with an example.
Asked: 2078 Poush; 2076 Baisakh | Marks: [4]

Answer

Join Ordering

When a query involves multiple joins, the order in which the joins are performed significantly affects performance. Different orderings produce different intermediate result sizes, and the total cost is the sum of the costs of all intermediate joins.

Why Join Ordering Matters

```
Example: r1 □ r2 □ r3

r1: 1000 tuples,  r2: 2000 tuples,  r3: 5 tuples

Option A: (r1 □ r2) □ r3
  Step 1: r1 ⋈ r2 → ~100,000 intermediate tuples (worst case)
  Step 2: 100,000 ⋈ r3 → ~50,000 result tuples
  High cost: large intermediate relation

Option B: (r1 □ r3) □ r2
  Step 1: r1 ⋈ r3 → ~5000 intermediate tuples (r3 is small!)
  Step 2: 5000 ⋈ r2 → manageable
  Lower cost: join with the small table first

Rule: Join with the most selective (smallest output) first.
```

Join Ordering Strategies

1. Greedy (Heuristic) Approach

- At each step, pick the join that produces the smallest intermediate result
- Not always optimal, but very fast to compute — $O(n^2)$ decisions
- Used in simple optimizers

2. Dynamic Programming (Optimal)

The DP approach computes the optimal join ordering for all subsets of relations, building up from smaller subsets to larger ones.

```
For n=3 relations: r1, r2, r3

Step 1: Best plan for singletons
  BestPlan({r1}) = scan r1 = b1 blocks
  BestPlan({r2}) = scan r2 = b2 blocks
  BestPlan({r3}) = scan r3 = b3 blocks

Step 2: Best plans for pairs
  BestPlan({r1,r2}): compare r1□r2 vs r2□r1; pick best
  BestPlan({r1,r3}): compare r1□r3 vs r3□r1; pick best
  BestPlan({r2,r3}): compare r2□r3 vs r3□r2; pick best

Step 3: Best plan for the full set
  BestPlan({r1,r2,r3}):
    Option A: BestPlan({r1,r2}) □ r3
    Option B: BestPlan({r1,r3}) □ r2
    Option C: BestPlan({r2,r3}) □ r1
    Pick option with minimum total cost

For n relations: 2^n subsets → DP runs in O(3^n) time
Practical for n ≤ 10-15 using left-deep trees only
```

General Principle: Choose Left-Deep Trees

Most optimizers only consider left-deep join trees (right operand is always a base relation). This reduces the search space from (2n-2)! to n! orderings and allows pipelined execution — the output of one join can be immediately consumed by the next.

Q: What is pipelining in query evaluation? Explain with an example. What are its advantages and limitations?

Asked: 2077 Chaitra (Regular Exam) | Marks: [4]

Answer

Pipelining in Query Evaluation

Pipelining (also called stream-based or on-the-fly evaluation) is a method of executing a query plan where the output tuples of one operation are immediately passed to the next operation in the pipeline, without writing intermediate results to disk. Multiple operations execute concurrently in a producer-consumer fashion.

Pipelining vs Materialization

Aspect	Pipelining	Materialization
Intermediate results	Passed directly between operators (in memory)	Written to disk between operators
Disk I/O	Minimal — no disk writes for intermediate results	High — each operator writes output to temp file
Memory	Requires buffers for each active operator	Needs only one operator's buffer at a time

When possible	When each tuple can be processed independently	When all input must be seen before any output (e.g., sort)
Example	Selection → Projection → Nested Loop Join	External Sort (must see all input before output)

Example: Pipelined Query Execution

```
Query: SELECT e.name, d.dname
       FROM Employee e JOIN Department d ON e.dept_id = d.dept_id
       WHERE e.salary > 50000

Query plan with pipelining:
   [1] Sequential scan of Employee (producer)
        ↓ (stream of tuples)
   [2] Selection: salary > 50000  (filter, passes matching tuples up)
        ↓ (stream of filtered tuples)
   [3] Indexed Nested Loop Join with Department  (outer = step 2 output)
        ↓ (stream of joined tuples)
   [4] Projection: name, dname  (projects on the fly)
        ↓
   Result to user

Steps 1-4 execute simultaneously in a pipeline.
No intermediate results are written to disk.
Each Employee tuple flows through all steps before the next is fetched.
```

Advantages of Pipelining

- Eliminates or minimizes disk I/O for intermediate results — the biggest performance benefit
- Reduces memory usage — each operator only needs a small buffer
- Results can be produced incrementally — user sees first result quickly
- Parallelism — multiple operators are active simultaneously
- Reduces overall query response time, especially for top-k or LIMIT queries

Limitations of Pipelining

- Blocking operators: Some operators cannot produce output until all input is consumed — e.g., Sort (must see all tuples before outputting), Hash Join (build phase must complete), Aggregation (GROUP BY must see all groups). These cannot be pipelined as the 'inner' consumer.
- Limited by slowest stage: The pipeline throughput is bounded by the slowest operator
- Complex implementation: Managing buffers and synchronization between operators is complex

Non-blocking (pipelineable) operators: Selection, Projection, Indexed NLJ (outer side), BNLJ (outer side)
Blocking operators (cannot pipeline as consumers): Sort, Hash Join (build phase), Aggregation, Duplicate elimination

SUMMARY TABLE — Chapter 5 Question Coverage

All Chapter 5 questions from TU-IOE BCT examination papers (2070–2082) and their coverage in this document.

No.	Question / Topic	Asked In	Marks	Status
C1	Query Processing Steps — scanner, parser, optimizer, code generator, query processor	All years (2070–2082)	[4]	Done
C2	Translating SQL to Relational Algebra & Query Trees	2080 Ashwin, 2079 Ashwin, 2073 Magh	[4]	Done
C3	Query Optimization — definition, types, why needed	2081 Ashwin, 2079 Jestha, 2074, 2072 Magh, 2070–2082	[4–6]	Done
C4	Cost Estimation — disk I/O model, relation statistics, buffer pool	2081 Chaitra, 2078 Baishakh, 2075 Bhadra, 2071 Magh, 2070 Magh	[4]	Done
C5	Heuristic Optimization — equivalence rules, rule-based approach	2082, 2079 Chaitra, 2078 Chaitra, 2075 Baisakh, 2073, 2072 Ashwin, 2071, 2070	[4–6]	Done
C6	Cost-Based Optimization — plan enumeration, dynamic programming	2081 Ashwin, 2079 Jestha, 2073 Magh, 2072 Magh, 2070 Magh	[4–6]	Done
A1	Selection Algorithms — linear scan, binary search, index scans	2078 Poush, 2074, 2072 Magh	[4]	Done
A2	Nested Loop Join (NLJ) — basic, block, indexed variants with cost	2081 Ashwin, 2080, 2077, 2076, 2075 Baisakh, 2073 Magh, 2072 Magh, 2071, 2070	[4–6]	Done
A3	Sort-Merge Join (SMJ) — algorithm, cost analysis	2081 Ashwin, 2080, 2079 Jestha, 2077, 2076, 2075 Bhadra, 2074, 2072 Ashwin, 2071, 2070	[4]	Done
A4	Hash Join — build/probe phases, cost analysis	2081 Ashwin, 2078 Baishakh, 2075 Bhadra, 2073 Magh, 2071 Magh	[4]	Done
C7	Join Ordering — left-deep trees, dynamic programming approach	2078 Poush, 2076	[4]	Done
C8	Pipelining in Query Evaluation	2077 Chaitra	[4]	Done

Notation used: b_r = blocks in relation r; n_r = tuples in r; M = buffer pages; h_i = B+ tree height; V(A,r) = distinct values of A in r. Cost metric is always disk block transfers unless otherwise noted. In exam answers, always define your notation before using it.

— End of Chapter 5 Answer Bank —